

COURSE CODE

SBT-DF203

COURSE TITLE

Basic Networking Skills for Digital Forensics

LAB TITLE

HTTP Analysis Using Wireshark - Embedded Image Traffic

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

10/09/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

Mr. Aminu Idris, AMCPN
Commandant

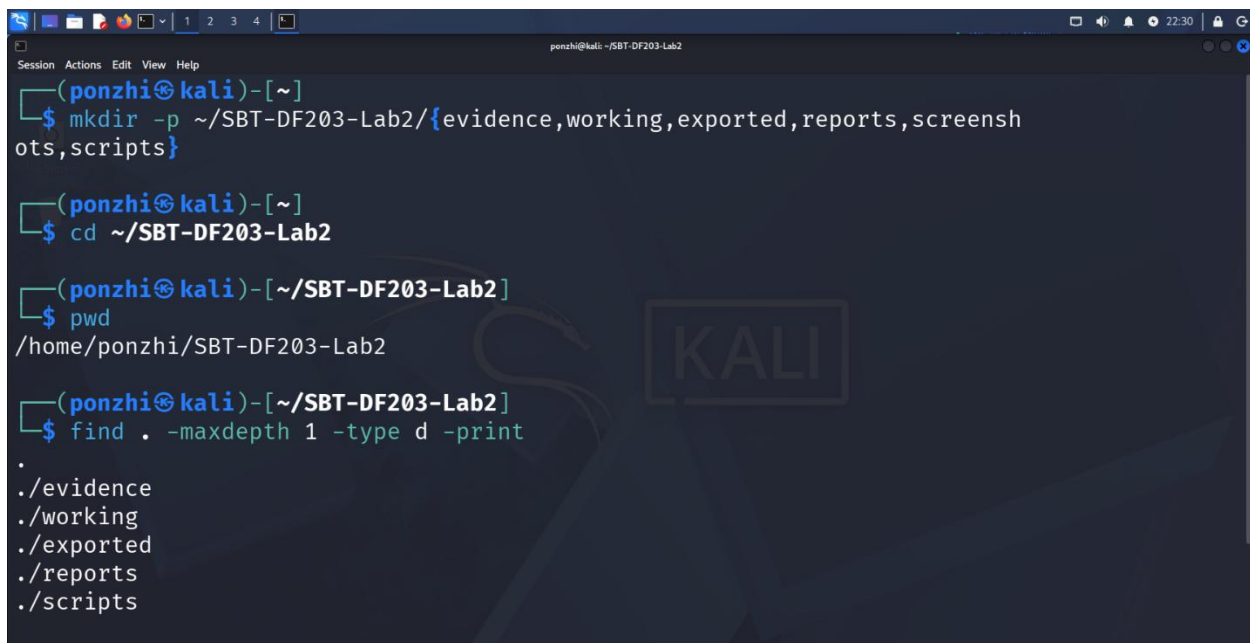
Executive Summary

This lab involved capturing a web browser's HTTP request to a locally hosted webpage (image.html) that included an image (lab_photo.png). The goal was to reconstruct the image from the captured data and check if the recovery was successful by comparing hashes. Additionally, we needed a second capture to compare how curl's default request behavior differs from a browser's.

The webpage request went through successfully (HTTP 200 OK). However, the web server denied access to the embedded image (HTTP 403 Forbidden) because the image's file permissions only allowed the file owner (root) to read it, which blocked the Apache service account. As a result, no image data was sent for recovery, and the only thing we were able to retrieve was the server's 403 error page instead of the original photo. This situation, along with the cause and suggested fixes, is documented below as we observed it.

Lab Folder Structure and Evidence Preparation

Tooling was already present on the analysis VM (apache2, curl, wireshark, tshark, imagemagick), and the Apache service was enabled to host the test page.



```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
(ponzhi@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}

(ponzhi@kali)-[~]
$ cd ~/SBT-DF203-Lab2

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ pwd
/home/ponzhi/SBT-DF203-Lab2

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ find . -maxdepth 1 -type d -print
.
./evidence
./working
./exported
./reports
./scripts
```

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo apt update
[sudo] password for ponzhi:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.7 MB]
Fetched 75.2 MB in 3min 38s (345 kB/s)
179 packages can be upgraded. Run 'apt list --upgradable' to see them.

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo apt install -y apache2 curl wireshark tshark imagemagick
apache2 is already the newest version (2.4.68-1).
curl is already the newest version (8.21.0-2).
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
imagemagick is already the newest version (8:7.1.2.27+dfsg1-1+b1).
imagemagick set to manually installed.
The following packages were automatically installed and are no longer required:
```

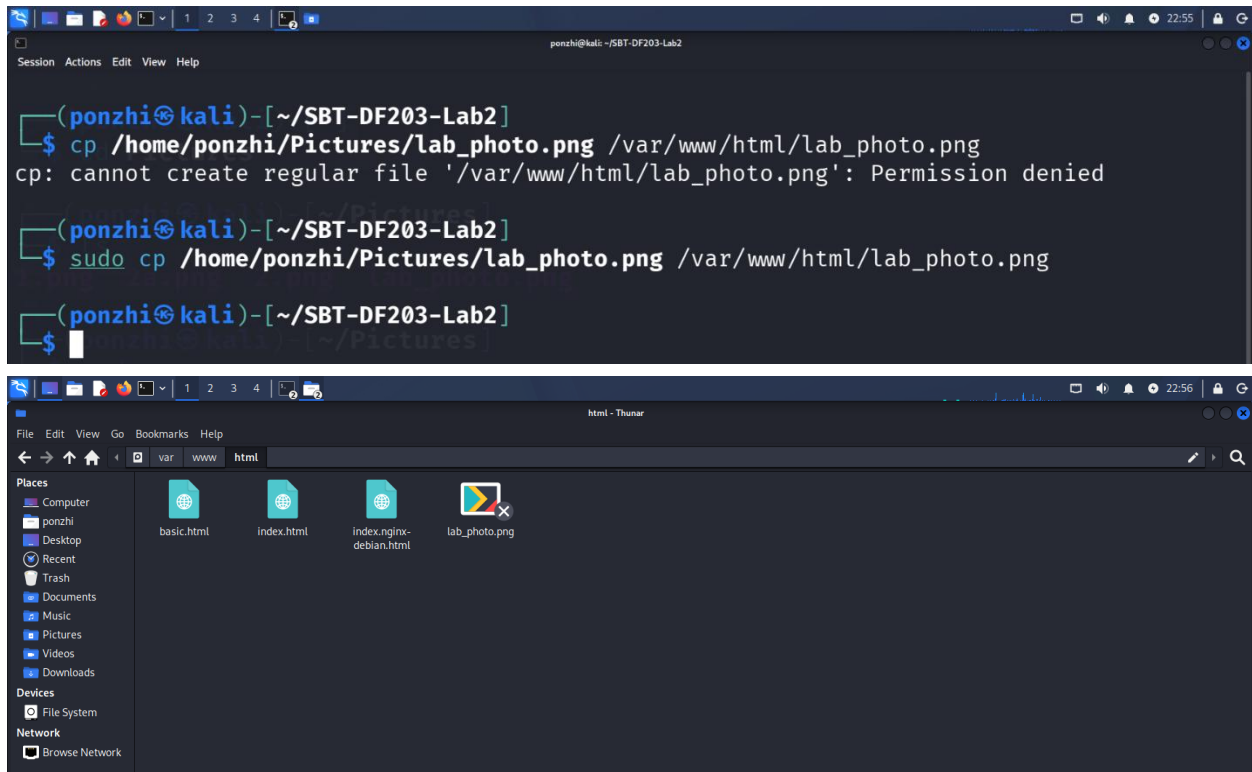
```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

apache2 is already the newest version (2.4.68-1).
curl is already the newest version (8.21.0-2).
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
imagemagick is already the newest version (8:7.1.2.27+dfsg1-1+b1).
imagemagick set to manually installed.
The following packages were automatically installed and are no longer required:
  bluez-hcidump      libhdf5-310      libpoppler156    python3.13
  firmware-marvell-prestera libhdf5-hl-310  libpython3.13-dev python3.13-dev
  iucode-tool        libntfs-3g89t64 libvidstab1.1    python3.13-minimal
  libabsl20260107    libopenjph0.27  libx265-215      python3.13-venv
  libbluray3         libperl5.40     perl-modules-5.40
Use 'sudo apt autoremove' to remove them.

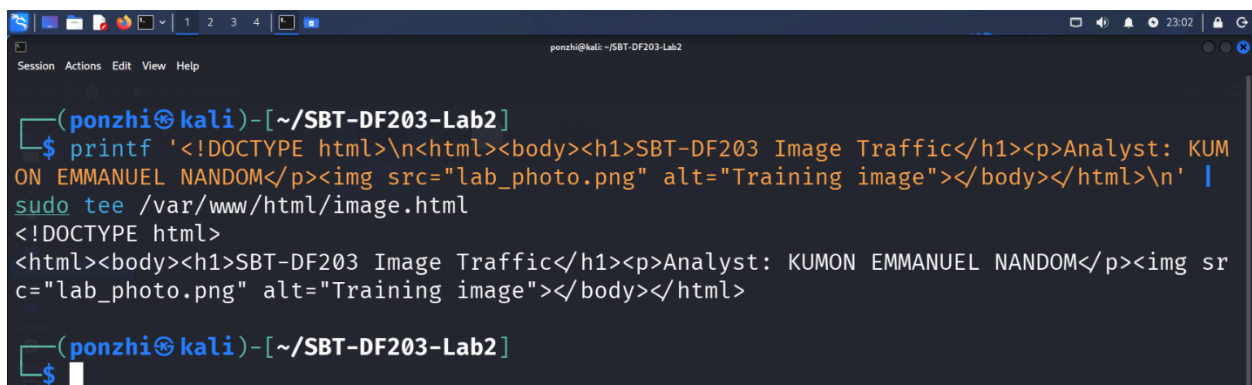
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 179

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo systemctl enable --now apache2
```

The source image was copied into the web root. The first attempt failed with "Permission denied"; the copy succeeded once elevated with sudo, which, as later analysis shows, left the file owned by root.



The test webpage image.html referencing the embedded image was then generated with a printf/tee one-liner and written into the web root.



Before generating any traffic, we recorded the SHA-256 hashes of both source objects to establish an integrity baseline. This baseline helps us compare the recovered objects later. When we tried to capture the hash of lab_photo.png without special permissions, we received a "Permission denied" error. This indicated an ownership issue. We were only able to successfully capture the hash by using sudo.

```
(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.png | tee reports/source_object_hashes.txt
sha256sum: /var/www/html/lab_photo.png: Permission denied
5c716fb63ef12489a7b0544f6d0d72db984fd74deaf13cee5c4b7c6aef1a9cfb /var/www/html/image.html

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo sha256sum /var/www/html/image.html /var/www/html/lab_photo.png | tee reports/source_object_hashes.txt
5c716fb63ef12489a7b0544f6d0d72db984fd74deaf13cee5c4b7c6aef1a9cfb /var/www/html/image.html
435652169074a824b3d719f5c672b4d5e9d27dc445970cd626d7643c7b5ca5fd /var/www/html/lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Part A — Verify the Webpage and Eliminate Cache Effects

File type, dimensions, and size of the source image were recorded before capture.

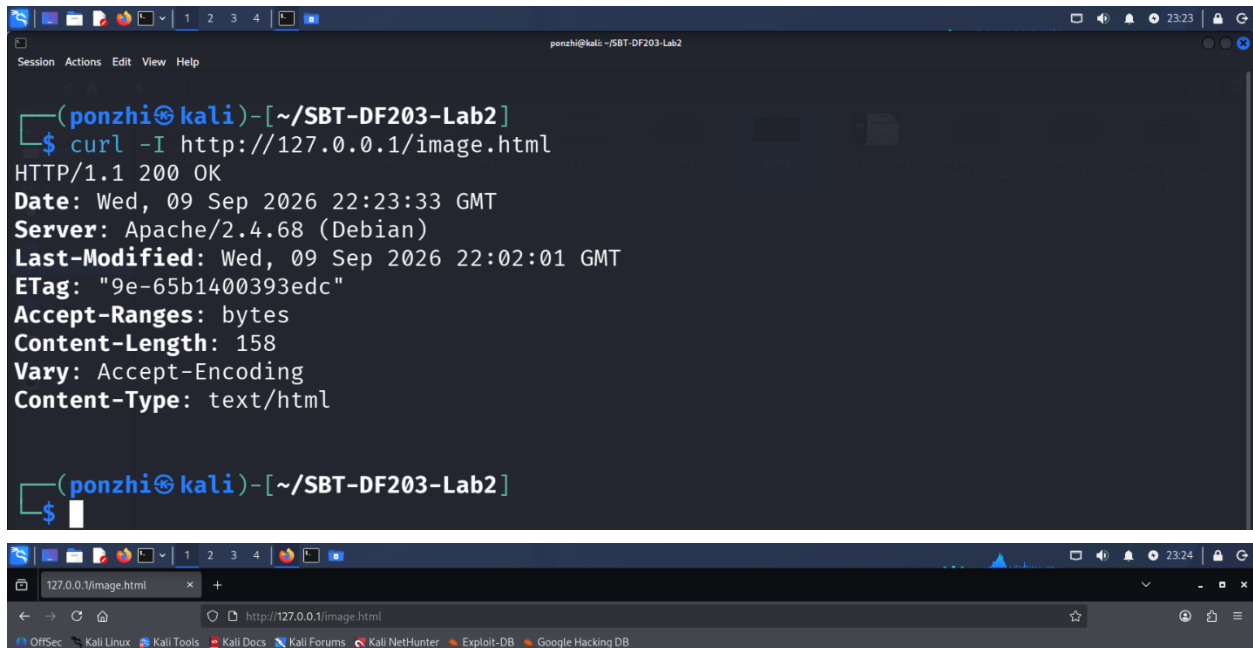
```
(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo file /var/www/html/lab_photo.png
/var/www/html/lab_photo.png: PNG image data, 1048 x 753, 8-bit/color RGBA, non-interlaced

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo identify /var/www/html/lab_photo.png
/var/www/html/lab_photo.png PNG 1048x753 1048x753+0+0 8-bit sRGB 923680B 0.000u 0:00.000

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.png
-rw-r--r-- 1 root root 158 Sep  9 23:02 /var/www/html/image.html
-rw----- 1 root root 903K Sep  9 22:55 /var/www/html/lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Note the permissions recorded: lab_photo.png is owned by root with mode -rw----- (owner read/write only), while image.html is world-readable (-rw-r--r--). A local curl HEAD check against the image confirms the consequence directly, before the browser capture was even started: the web server already returns 403 Forbidden for the image object.



```
(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Wed, 09 Sep 2026 22:23:33 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Wed, 09 Sep 2026 22:02:01 GMT
ETag: "9e-65b1400393edc"
Accept-Ranges: bytes
Content-Length: 158
Vary: Accept-Encoding
Content-Type: text/html

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

127.0.0.1/image.html

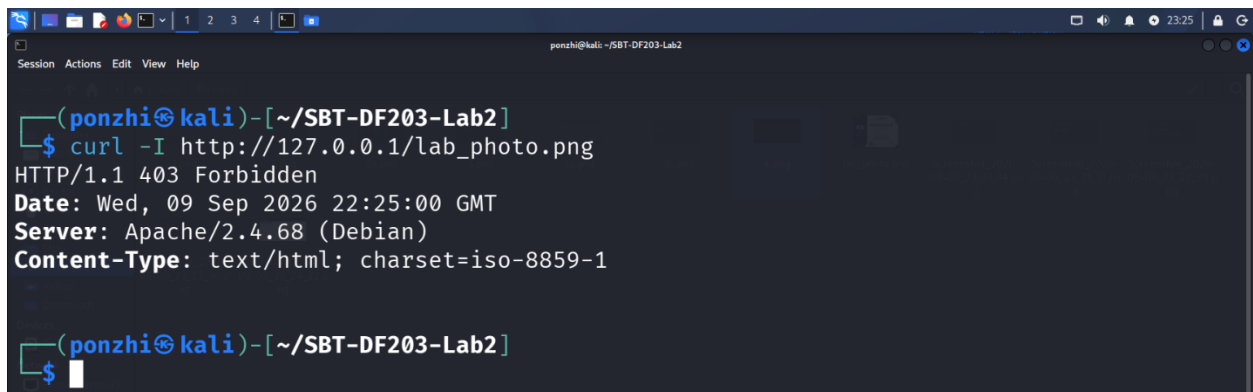
http://127.0.0.1/image.html

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

SBT-DF203 Image Traffic

Analyst: KUMON EMMANUEL NANDOM

Training image

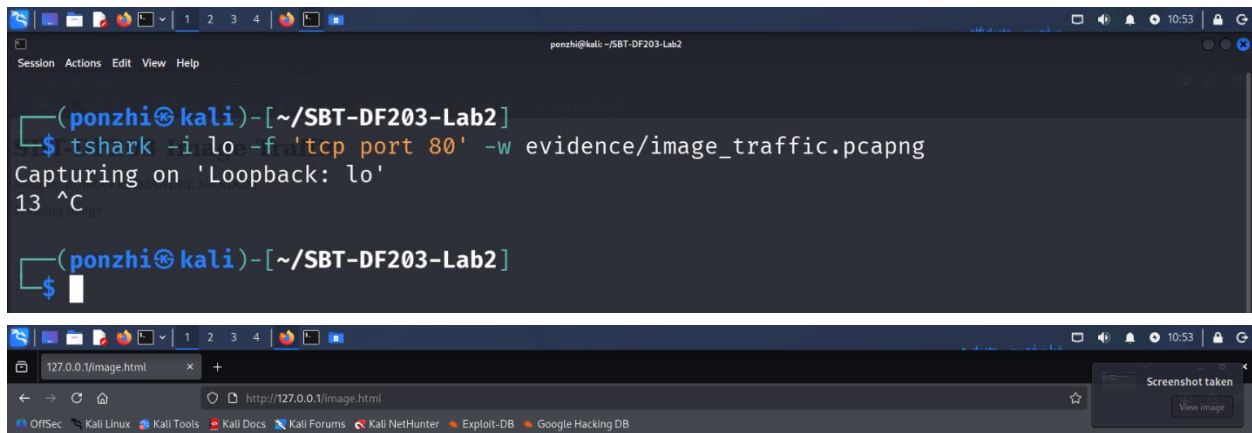


```
(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/lab_photo.png
HTTP/1.1 403 Forbidden
Date: Wed, 09 Sep 2026 22:25:00 GMT
Server: Apache/2.4.68 (Debian)
Content-Type: text/html; charset=iso-8859-1

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Part B — Capture Browser-Generated HTTP Traffic

A live capture was started on the loopback interface, filtered to tcp port 80, immediately before browsing to `http://127.0.0.1/image.html` in a private window. Thirteen packets were captured before the process was stopped with Ctrl+C.

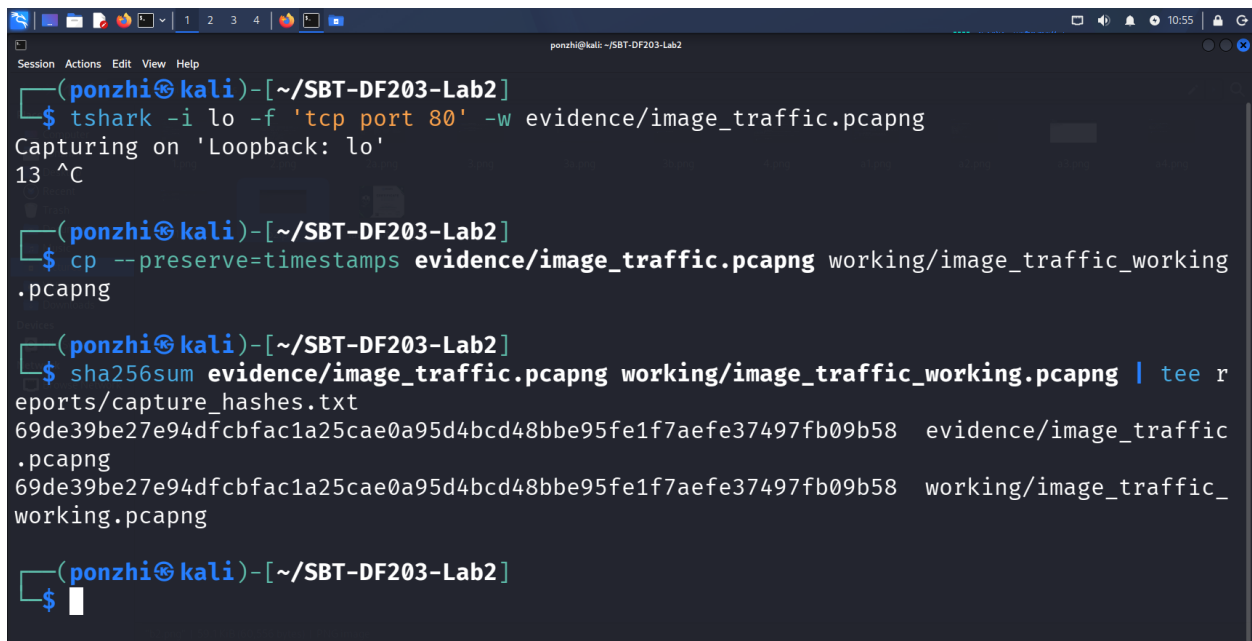


SBT-DF203 Image Traffic

Analyst: KUMON EMMANUEL NANDOM

Training Image

We created a working copy of the capture for analysis and kept the original as read-only evidence. The SHA-256 hashes of both the evidence file and the working copy are the same, which shows that the working copy is an exact duplicate.



Part C — Prove That Two Objects Were Requested

Filtering on `http.request` confirms the browser issued exactly two GET requests on the same TCP stream (stream 0) one for `/image.html` and one for `/lab_photo.png`, roughly 50 ms apart.

```
(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.
dstport \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv
4      2026-09-10T10:53:16.284295303+0100      0      127.0.0.1      43144      127.0.0.
1      80      GET      /image.html      127.0.0.1
8      2026-09-10T10:53:16.334867562+0100      0      127.0.0.1      43144      127.0.0.
1      80      GET      /lab_photo.png      127.0.0.1

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Filtering for `http.response` shows two responses: the HTML page returned a status of 200 OK. In contrast, the image request returned a status of 403, with a content type of `text/html; charset=iso-8859-1` and a body size of 314 bytes. This means it returned an HTML error page instead of image data.

```
(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type
-e http.content_length \
  | tee reports/http_object_responses.tsv
6      2026-09-10T10:53:16.295264321+0100      0      200      text/html      160
9      2026-09-10T10:53:16.335330503+0100      0      403      text/html; charset=iso-8
859-1      314

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Part D — TCP Segmentation and Reassembly Analysis

A combination of an HTTP filter and a specific HTTP request extraction shows the same results as Part C: frame 4 (GET /image.html, tcp.stream 0) and frame 8 (GET /lab_photo.png, tcp.stream 0). Frame 6 received a 200 OK response, while frame 9 received a 403 Forbidden response.

```
ponzhi@kali: ~/SBT-DF203-Lab2
Number of stat entries = 1
Number of packets = 13

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http' | head -20
 4 0.003571008      127.0.0.1 → 127.0.0.1      HTTP 599 GET /image.html HTTP/1.1
 6 0.014540026      127.0.0.1 → 127.0.0.1      HTTP 562 HTTP/1.1 200 OK (text/html)
 8 0.054143267      127.0.0.1 → 127.0.0.1      HTTP 509 GET /lab_photo.png HTTP/1.1
 9 0.054606208      127.0.0.1 → 127.0.0.1      HTTP 596 HTTP/1.1 403 Forbidden (text/htm
l)

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields -e frame.
number -e tcp.stream -e http.request.uri
4      0      /image.html
8      0      /lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

The lab instructions require us to isolate the TCP segments that make up the image response using the filter `tcp.stream==STREAM && tcp.len>0`. However, looking for data in stream index 8 (instead of the existing stream at index 0) gives no results, because there were never 9 active streams — this single browser session only produced `tcp.stream 0`.

```
ponzhi@kali: ~/SBT-DF203-Lab2

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==8 && tcp.len>0' -T fi
elds \
  -e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ac
k \
  | tee reports/image_stream_segments.tsv

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

It's important to note that there was no TCP-level reassembly to analyze for the image object, even with the correct stream. The 403 response was contained entirely within a single 596-byte frame. This means that Wireshark's "Reassembled TCP Segments" indicator did not show up for this transaction. The summary of the TCP conversation shows that the entire browser interaction—including the SYN/ACK handshake, both requests, both responses, and the closing ACKs—only involved 13 frames and 2,876 bytes in total.

```

k \
| tee reports/image_stream_segments.tsv

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt

TCP Conversations
Filter:<No Filter>

  | |      Total      |      Relative      |      Duration      |      |      |      |      |
  | |      Bytes      |      Start         |      |      |      |      |      |
127.0.0.1:43144 ↔ 127.0.0.1:80      6 1,430 bytes      7 1,
446 bytes      13 2,876 bytes      0.000000000      5.0606

```

Part E — Export the Embedded Image

We used tshark's --export-objects http option to directly extract HTTP objects from the working copy. These objects were saved in the exported/http_objects folder.

```

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ mkdir -p exported/http_objects

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects

1 0.000000000      127.0.0.1 → 127.0.0.1      TCP 74 43144 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=169263725 TSecr=0 WS=512
2 0.000026382      127.0.0.1 → 127.0.0.1      TCP 74 80 → 43144 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3553166349 TSecr=169263725 WS=512
3 0.000035145      127.0.0.1 → 127.0.0.1      TCP 66 43144 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=169263725 TSecr=3553166349
4 0.003571008      127.0.0.1 → 127.0.0.1      HTTP 599 GET /image.html HTTP/1.1
5 0.003751434      127.0.0.1 → 127.0.0.1      TCP 66 80 → 43144 [ACK] Seq=1 Ack=534 Win=65024 Len=0 TSval=3553166353 TSecr=169263729
6 0.014540026      127.0.0.1 → 127.0.0.1      HTTP 562 HTTP/1.1 200 OK (text/html)
7 0.014573011      127.0.0.1 → 127.0.0.1      TCP 66 43144 → 80 [ACK] Seq=534 Ack=497 Win=65536 Len=0 TSval=169263740 TSecr=3553166364

```

Two files were recovered, image.html and an object named lab_photo.png

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ find exported/http_objects -maxdepth 1 -type f -ls
 2491899    4 -rw-r--r--    1 ponzhi  ponzhi      158 Sep 10 12:08 exported/http_ob
jects/image.html
 2491900    4 -rw-r--r--    1 ponzhi  ponzhi      314 Sep 10 12:08 exported/http_ob
jects/lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Running file(1) on both recovered objects shows that the size difference is not a coincidence. The exported "lab_photo.png" is actually identified as an HTML document (ASCII text) instead of image data. It turns out to be the 403 Forbidden error page returned by Apache, which Wireshark saved under the filename of the requested object.

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ find exported/http_objects -maxdepth 1 -type f -ls
 2491899    4 -rw-r--r--    1 ponzhi  ponzhi      158 Sep 10 12:08 exported/http_ob
jects/image.html
 2491900    4 -rw-r--r--    1 ponzhi  ponzhi      314 Sep 10 12:08 exported/http_ob
jects/lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ file exported/http_objects/*
exported/http_objects/image.html:  HTML document, ASCII text
exported/http_objects/lab_photo.png: HTML document, ASCII text

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Final hash comparison against the source objects confirms this conclusively: image.html's hash matches exactly, while the exported lab_photo.png hash does not match the original image's hash at all.

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
d_object_hashes.txt
sha256sum: /var/www/html/lab_photo.png: Permission denied
5c716fb63ef12489a7b0544f6d0d72db984fd74deaf13cee5c4b7c6aef1a9cfb  exported/http_objects/
image.html
4c4c2fe884ab5b833d6a7ccae59106c2eb9659f15ea06a1c29ef3d7322d656e0  exported/http_objects/
lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo sha256sum /var/www/html/lab_photo.png exported/http_objects/* | tee reports/ext
racted_object_hashes.txt
435652169074a824b3d719f5c672b4d5e9d27dc445970cd626d7643c7b5ca5fd  /var/www/html/lab_phot
o.png
5c716fb63ef12489a7b0544f6d0d72db984fd74deaf13cee5c4b7c6aef1a9cfb  exported/http_objects/
image.html
4c4c2fe884ab5b833d6a7ccae59106c2eb9659f15ea06a1c29ef3d7322d656e0  exported/http_objects/
lab_photo.png

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Part F — Compare curl and Browser Behaviour

A capture process started in the background for the curl test, followed by a curl request to image.html. The background capture failed to save because it was started with sudo, which meant it ran as root. Since it couldn't write to the evidence/ directory owned by a regular user, it produced a "Permission denied" error and stopped before capturing any packets. However, the curl request was executed, and its request and response details were captured using curl's verbose output.

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
[1] 170637
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
tshark: The file to which the capture would be saved ("evidence/curl_only.pcapng") could
not be opened: Permission denied.
[1] + exit 1 sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_onl
y.pcapn

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 48004
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
```

The detailed curl log shows one request and response: curl made a GET request for /image.html and received a 200 OK response with a Content-Length of 158, which matches the size of image.html.

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 12:30:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Wed, 09 Sep 2026 22:02:01 GMT
< ETag: "9e-65b1400393edc"
< Accept-Ranges: bytes
< Content-Length: 158
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [158 bytes data]
100 158 100 158 0 0 4756 0
* Connection #0 to host 127.0.0.1:80 left intact
```

The background capture file was never created. As a result, the next attempt to extract requested-object URIs from evidence/curl_only.pcapng fails with the message.

```
ponzhi@kali: ~/SBT-DF203-Lab2
Session Actions Edit View Help
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Wed, 09 Sep 2026 22:02:01 GMT
< ETag: "9e-65b1400393edc"
< Accept-Ranges: bytes
< Content-Length: 158
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [158 bytes data]
100 158 100 158 0 0 9401 0
* Connection #0 to host 127.0.0.1:80 left intact

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$ tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri
tee reports/curl_requested_objects.txt
tshark: The file "evidence/curl_only.pcapng" doesn't exist.

(ponzhi@kali)-[~/SBT-DF203-Lab2]
$
```

Mini Evidence and Chain-of-Custody Worksheet

Case/lab identifier	SBT-DF203-Lab2-KumponEmmanuelNandom
Trainee name	Kumpon Emmanuel Nandom
Date and time started	9th September 2026 by 22:50 pm captured and continued on 10th September 2026 at 10:53 am
Evidence file name(s)	evidence/image_traffic.pcapng evidence/curl_only.pcapng
Source or generation method	Browser-generated HTTP capture on loopback interface
Original SHA-256	image.html: 5c716fb63ef12489a7b0544f6d0d72db984deaf13cee5c4b7c6aef1a9cfb lab_photo.png: 435652169074a824b3d719f5c672b4d5e9d27dc445970cd626d7b5ca5fd
Working-copy SHA-256	image_traffic_working.pcapng: 69de39be27e94dfcbfac1a25cae0a95d4bcd48bbe95fe1f7aefe37497fb09b58
Analysis workstation/VM	Kali Linux
Notes on any changes	The file lab_photo.png was copied using sudo, which made it owned by the root user and set its permissions to 600. This means only the owner can read it, and the Apache service account could not access it. As a result, every request for the image resulted in a 403 Forbidden error, which is the main reason for the failed object recovery in Part E. Additionally, the curl-only capture in Part F also failed to save because of a permission issue with the evidence/ directory.

Required Forensic Findings

Finding	Student Observation
Number of HTTP requests	2 (GET /image.html, GET /lab_photo.png), both on tcp.stream 0
HTML request URI and timestamp	/image.html — frame 4, 2026-09-10 10:53:16.284
Image request URI and timestamp	/lab_photo.png — frame 8, 2026-09-10 10:53:16.335
HTTP response codes	200 OK for /image.html, 403 Forbidden for /lab_photo.png
Image content type	None returned — the image request was denied; the response content type was text/html; charset=iso-8859-1 (Apache's 403 error page), not an image MIME type
Reported Content-Length	image.html: 158 bytes ; lab_photo.png error response: 314 bytes
Number of reassembled TCP segments	
Sum of TCP payload lengths	2,876 bytes total across the one TCP conversation
Original image SHA-256	435652169074a824b3d719f5c672b4d5e9d27dc445970cd626d7643c7b5ca5fd
Extracted image SHA-256	4c4c2fe884ab5b833d6a7ccae59106c2eb9659f15ea06a1c29ef3d7322d656e0
Hash comparison conclusion	The recovered file "lab_photo.png" is not the original image. Instead, it is an HTML document that is 314 bytes in size, which is actually a 403 error page. The original PNG file should be 903 KB. The only file that matched the original was image.html, with the hash 5c716fb6...a9cfb.
curl/browser difference	We could not verify the packet trace because the file evidence/curl_only.pcapng could not be saved. The root-owned tshark process was denied write access to the evidence/ directory. The curl -v log shows that curl made one GET request for image.html, which received a response of 200 OK and a Content-Length of 158. There was no follow-up request for the embedded image. This behavior is normal for curl, as it does not parse HTML or automatically fetch linked resources, unlike a web browser.